

# Sprint 2 — Integração com Middleware Orientado a Mensagens

## 1. Vídeo de demonstração dos eventos de order.created e order.accepted

https://youtu.be/fp4ePh544T8

## 2. Documentação dos Eventos

O sistema publica **cinco eventos** que cobrem todo o ciclo de vida do pedido, do momento da criação até a entrega final. Cada transição de status no `OrderService` dispara um evento correspondente, consumido por workers dedicados que persistem notificações in-app.

Visão geral

| Campo              | order.created                              | order.accepted                                                               | order.delivery_assigned                                                      | order.in_delivery                           | order.c                    |
|--------------------|--------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------|----------------------------|
| Produtor           | OrderService.create()                      | OrderService.updateStatus()                                                  | OrderService.updateStatus()                                                  | OrderService.updateStatus()                 | OrderSe                    |
| Transição          | (criação) → CRIADO                         | CRIADO → ACEITO                                                              | ACEITO → ENTREGADOR_DESIGNADO                                                | ENTREGADOR_DESIGNADO → EM_ENTREGA           | EM_ENTF                    |
| Quem dispara       | Cliente                                    | Restaurante                                                                  | Entregador                                                                   | Entregador                                  | Entrega                    |
| Filas consumidoras | restaurant_notifications                   | client_notifications, delivery_notifications                                 | restaurant_notifications, client_notifications                               | client_notifications                        | restaur client_            |
| Exchange           | uaikifome (topic)                          | uaikifome (topic)                                                            | uaikifome (topic)                                                            | uaikifome (topic)                           | uaikifo                    |
| Routing key        | order.created                              | order.accepted                                                               | order.delivery_assigned                                                      | order.in_delivery                           | order.c                    |
| Ação do consumidor | Notifica dono do restaurante (novo pedido) | Notifica cliente (pedido aceito) e todos os entregadores (pedido disponível) | Notifica restaurante (entregador a caminho) e cliente (entregador designado) | Notifica cliente (pedido saiu para entrega) | Notifica finalizad entregu |

Payloads

Todos os payloads carregam o campo `type` (espelhando a routing key), o que permite que uma mesma fila trate múltiplos eventos diferenciando a mensagem pelo `type` do payload.

order.created

```
{
  "type": "order.created",
  "orderId": "c2d8cb00-fb11-464e-b3f1-bbb8677a8863",
  "restaurantId": "15af2600-de45-4144-8a56-1111da841275",
  "clientId": "a3f1cc90-12ab-4e77-b902-4321fa098765"
}
```

order.accepted

```
{
  "type": "order.accepted",
  "orderId": "c2d8cb00-fb11-464e-b3f1-bbb8677a8863",
  "clientId": "a3f1cc90-12ab-4e77-b902-4321fa098765"
}
```

order.delivery\_assigned

```
{
  "type": "order.delivery_assigned",
  "orderId": "c2d8cb00-fb11-464e-b3f1-bbb8677a8863",
  "clientId": "a3f1cc90-12ab-4e77-b902-4321fa098765",
  "restaurantId": "15af2600-de45-4144-8a56-1111da841275"
}
```

order.in\_delivery

```
{
  "type": "order.in_delivery",
  "orderId": "c2d8cb00-fb11-464e-b3f1-bbb8677a8863",
}
```

```
"clientId": "a3f1cc90-12ab-4e77-b902-4321fa098765"
}
```

#### order.delivered

```
{
  "type": "order.delivered",
  "orderId": "c2d8cb00-fb11-464e-b3f1-bbb8677a8863",
  "clientId": "a3f1cc90-12ab-4e77-b902-4321fa098765",
  "restaurantId": "15af2600-de45-4144-8a56-1111da841275"
}
```

### Mensagens geradas por evento

#### Restaurante (restaurant\_notifications):

- `order.created` → "Novo pedido recebido"
- `order.delivery_assigned` → "Entrega aceita — entregador a caminho da retirada"
- `order.delivered` → "Pedido finalizado"

#### Cliente (client\_notifications):

- `order.accepted` → "Seu pedido foi aceito pelo restaurante"
- `order.delivery_assigned` → "Um entregador foi designado e está indo retirar seu pedido"
- `order.in_delivery` → "Seu pedido saiu para entrega"
- `order.delivered` → "Pedido entregue!"

#### Entregadores (delivery\_notifications):

- `order.accepted` → "Novo pedido disponível para entrega" (broadcast para todos)

### Campo `userId` na tabela `notifications`

O campo `userId` armazena o ID de **quem recebe** a notificação, não de quem gerou o evento.

Exemplo para `order.created`: o evento é gerado pelo cliente ao criar um pedido, mas a notificação é destinada ao dono do restaurante. O consumer busca a entidade `Restaurant` pelo `restaurantId` do payload, extrai o `restaurant.userId` (ID do usuário dono) e persiste esse valor. Assim, quando o restaurante faz `GET /notifications`, o endpoint filtra por `userId = req.user.id` e retorna apenas as notificações destinadas a ele.

### Filas e bindings

As filas são nomeadas por **destinatário**, não por evento — uma mesma fila pode receber múltiplas routing keys, e o consumer escolhe a mensagem pelo `type` do payload.

| Fila                                  | Exchange               | Routing keys vinculadas                                                                                                            | Destinatário                      |
|---------------------------------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>restaurant_notifications</code> | <code>uaikifome</code> | <code>order.created</code> , <code>order.delivery_assigned</code> , <code>order.delivered</code>                                   | Usuário dono do restaurante       |
| <code>client_notifications</code>     | <code>uaikifome</code> | <code>order.accepted</code> , <code>order.delivery_assigned</code> , <code>order.in_delivery</code> , <code>order.delivered</code> | Cliente do pedido                 |
| <code>delivery_notifications</code>   | <code>uaikifome</code> | <code>order.accepted</code>                                                                                                        | Todos os entregadores (broadcast) |

## 2. Relatório de Integração

### Ferramenta escolhida: RabbitMQ

O RabbitMQ foi escolhido por ser o MOM mais consolidado no mercado para comunicação assíncrona orientada a eventos, com suporte nativo a múltiplos padrões de roteamento, painel de gerenciamento visual (Management UI) e client oficial para Node.js (`amqplib`). Sua configuração via Docker é simples e seu modelo de exchanges e filas se encaixa diretamente na arquitetura do projeto.

### Padrão utilizado: Topic Exchange

Optou-se pelo padrão **Topic Exchange** em vez de Direct Exchange ou Fanout por três razões:

1. **Um único exchange** (`uaikifome`) centraliza todos os eventos do domínio, facilitando o gerenciamento;
2. **Routing keys expressivas** (`order.created`, `order.accepted`, `order.delivery_assigned`, `order.in_delivery`, `order.delivered`) tornam o fluxo auto-documentado;
3. **Extensibilidade**: novos consumidores podem se inscrever em padrões como `order.*` sem modificar os produtores, e uma mesma fila pode receber múltiplas routing keys sem reestruturação.

As mensagens são configuradas como `persistent: true`, garantindo que não sejam perdidas em caso de reinício do broker.

### Arquitetura de integração

O worker (`OrderConsumer`) é inicializado junto ao servidor Node.js, após a conexão com o banco de dados ser estabelecida. Ele registra handlers assíncronos nas três filas. O produtor (`OrderPublisher`) é chamado dentro dos services após a persistência no banco, com `try/catch` isolado para que uma falha no RabbitMQ não interrompa a resposta REST ao cliente.

A conexão com o RabbitMQ é gerenciada como singleton (`getRabbitMQChannel`) com lógica de retry (5 tentativas, intervalo de 3 segundos), necessária porque o container RabbitMQ demora alguns segundos a mais para inicializar em relação ao backend.

Tanto o produtor quanto o consumidor produzem **logs detalhados** (`[producer]` e `[consumer:<fila>]`) que registram timestamp, exchange, routing key, payload completo, destinatário resolvido e o resultado (`ack/nack`), facilitando a observabilidade do fluxo de mensagens.

#### Desafios encontrados

**RabbitMQ não disponível na inicialização:** o backend inicializava antes do container RabbitMQ estar pronto, causando erro de handshake. Resolvido com lógica de retry na função `getRabbitMQChannel`.

**userId vs restaurantId:** o payload dos eventos destinados ao restaurante (`order.created`, `order.delivery_assigned`, `order.delivered`) carrega o `restaurantId` (ID da entidade `Restaurant`), não o ID do usuário dono. O consumer precisou buscar a entidade `Restaurant` para obter o `userId` correto antes de persistir a notificação.

**Múltiplos eventos por fila:** com a expansão para cinco eventos, as filas `restaurant_notifications` e `client_notifications` passaram a receber múltiplas routing keys. Para evitar criar uma fila por evento, o payload de cada mensagem inclui o campo `type`, e os consumidores usam mapas (`RESTAURANT_MESSAGES`, `CLIENT_MESSAGES`) que associam cada `type` à mensagem correta da notificação.

---

### 3. Comprovação de Funcionamento

#### Painel de gerenciamento do RabbitMQ

O Management UI do RabbitMQ confirma a infraestrutura de integração ativa: **1 exchange** (`uaikifome`), **3 filas** (`restaurant_notifications`, `client_notifications`, `delivery_notifications`) e **3 consumidores** registrados, correspondentes aos workers da aplicação. O gráfico de *Message rates* registra a publicação e entrega das mensagens, e a métrica *Queued messages* zerada após o processamento comprova que as mensagens foram consumidas e confirmadas (ack) com sucesso:

 Painel de gerenciamento do RabbitMQ com filas e consumidores ativos